Kruskal's Algorithm for Communication Tower Network

Minimum Spanning Tree (MST) with Edge Update, Running Cost and Disconnected Graph Detection (C++)

1. Problem Statement

Develop a C++ program to represent communication towers as vertices and candidate communication links as weighted edges in a graph. Apply Kruskal's algorithm to connect all communication towers at minimum total cost by constructing a Minimum Spanning Tree (MST). Display the selected links, their weights, the total cost, and the average weight of the selected links.

Suggested Modifications

- **A)** Modify the program to update the weight of an existing edge and reapply Kruskal's algorithm to generate the updated MST.
- **B)** Modify the program to display every selected edge along with its weight and the running total cost after each edge is added to the MST.
- **C)** Modify the program to detect a disconnected graph and report whether a spanning tree can be formed. If the graph is disconnected, display a message indicating that a spanning tree cannot be formed.

2. Objective

- To represent a communication network as a weighted graph (towers = vertices, links = edges).
- To apply Kruskal's greedy algorithm with Union-Find cycle detection to build the MST using exactly V - 1 links.
- To compute the total cost and average weight of the selected links.
- To update a link weight and regenerate the MST, showing how the network design changes (Modification A).
- To display the running total cost after each edge is added (Modification B).
- To detect a disconnected network and report that no spanning tree exists (Modification C).

3. Algorithm

3.1 Basic Kruskal's Algorithm

Step 1: Start the program.

Step 2: Input the number of communication towers (V) and candidate links (E).

Step 3: Assign a weight to each link (installation cost or distance).

Step 4: Sort all edges in ascending order of weight.

Step 5: Initialize the MST as empty and set total cost = 0.

Step 6: Select the minimum-weight edge from the sorted list.

Step 7: Check whether adding the edge creates a cycle. If no cycle, add it to the MST and update the total cost. If a cycle forms, discard the edge.

Step 8: Repeat Steps 6 and 7 until V - 1 edges have been selected.

Step 9: Display the selected links and their weights.

Step 10: Calculate and display the total cost and average weight = Total cost / (V - 1).

Step 11: Stop the program.

3.2 Changes for the Modifications

Modification B (running total): In Step 7, every time an edge is added to the MST, print the step number, the link, its weight and the updated total cost immediately.

Modification C (disconnected graph): After the loop ends (all edges examined), if the number of selected edges is less than V - 1, display 'Graph is disconnected. Spanning tree cannot be formed.' and the number of separate groups (components = V - edges selected). Otherwise display the total cost and average weight.

Modification A (update edge weight):

A1. After displaying the MST, ask the user whether to update an edge weight.

A2. If yes, input the two towers and the new weight.

A3. Search the edge list for the link between the two towers (in either direction). If it does not exist, display 'Link does not exist'.

A4. If found, replace its weight with the new weight.

A5. Re-sort the edge list and apply Kruskal's algorithm again to generate the updated MST (with Modifications B and C).

A6. Repeat until the user chooses not to update any more edges.

4. Flowchart

Flowchart of Kruskal's algorithm for the communication tower network, including Modification A (update edge weight), Modification B (display running total cost) and Modification C (detect disconnected graph).

